

CSE 493s HW 1

Vincent Chau, Kevin Yin
vkchau, kevin30

1st May, 2026

Problem 0: Train Infrastructure Setup

Please refer to <https://api.wandb.ai/links/vincentczhou-university-of-washington/qx1o7exe> for the report.

Problem 1: Training on Algorithmic Tasks

Please refer to <https://api.wandb.ai/links/vincentczhou-university-of-washington/qx1o7exe> for the report.

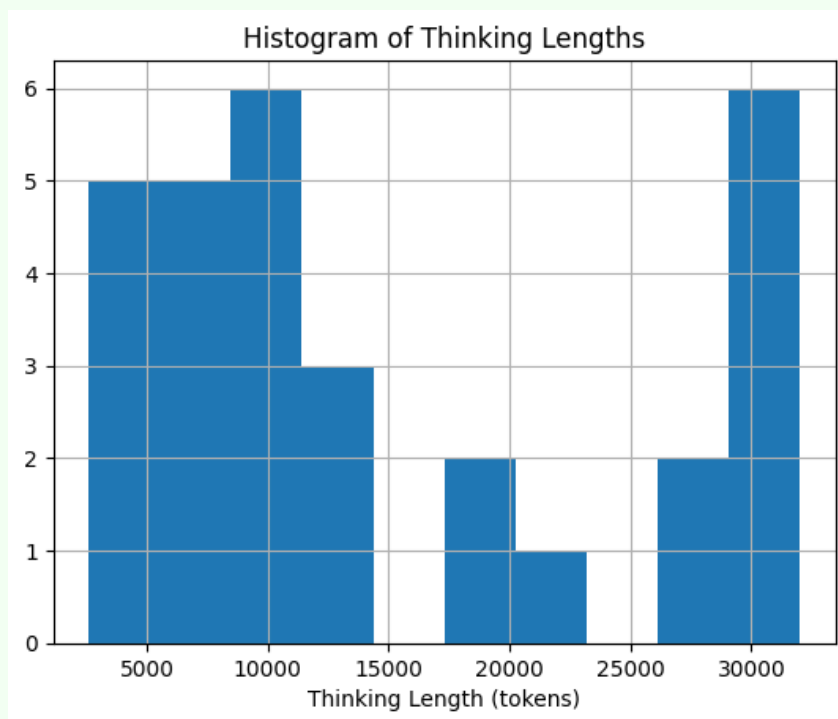
Problem 2: Test-Time Scaling

2.1 Warm-up

Using the Qwen3-4B model with `MAX_NEW_TOKENS = 32,000`, the accuracies are as follows:

- Accuracy with thinking, `exact_match`: 66.67%
- Accuracy with thinking, `flexible_extract`: 66.67%
- Accuracy without thinking, `exact_match`: 20%
- Accuracy without thinking, `flexible_extract`: 20%

Even with `MAX_NEW_TOKENS = 32,000`, there were 6 problems where the model did not finish thinking (there was no `</think>` token generated). For all of these problems, the model's `exact_match` and `flexible_extract` answers were incorrect, and the thinking lengths were counted as 32,000 (31,999 if not including the initial `<think>` token).

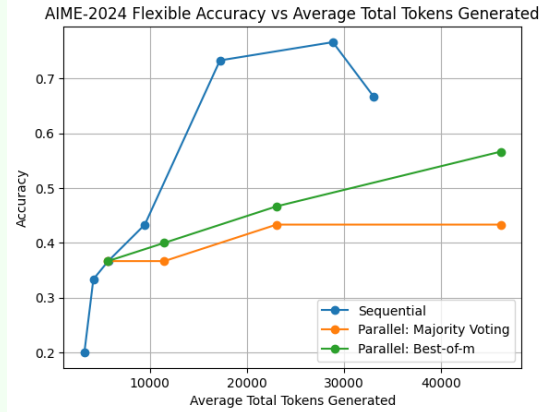
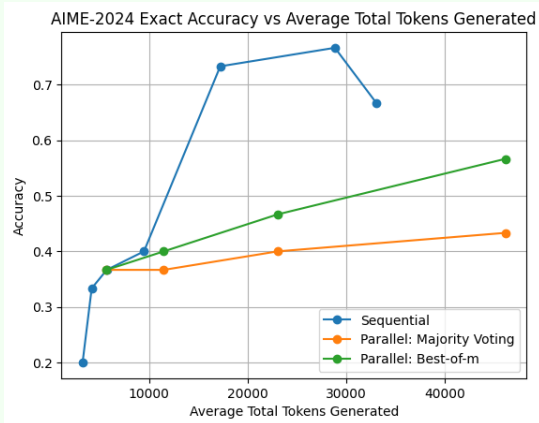
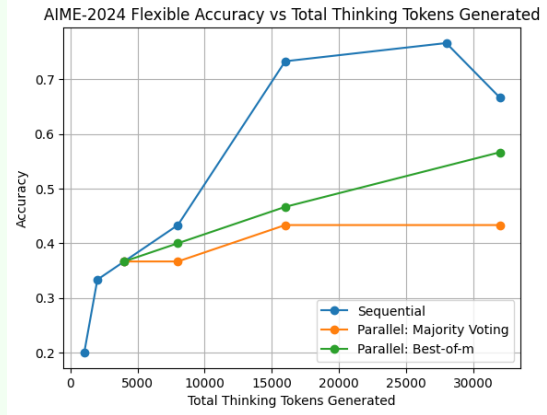
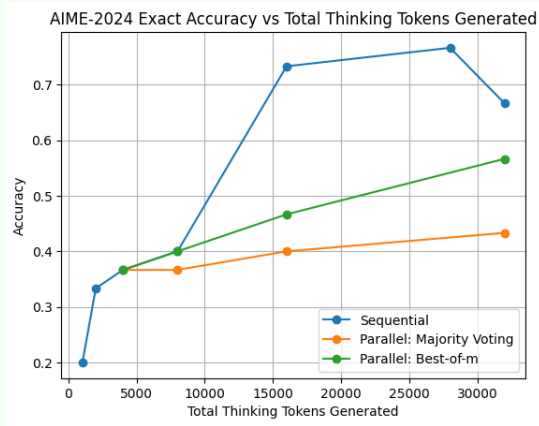


2.2 Scaling Experiments

If we care about total compute, then sequential scaling is the better scaling strategy. The plots show that as the thinking budget, a rough proxy for compute, increases, sequential scaling outperforms parallel scaling in accuracy.

If we care about latency, then parallel scaling with majority voting is the better scaling strategy. This is because independent completions can be generated at the same time, and the plots show that accuracy can still improve with parallel scaling. For example, with $m = 8$ which is a thinking budget of $8 \times 4000 = 32000$ tokens, the responses can be generated in parallel with the same latency as generating one sequential response with a thinking budget of 4000 tokens, but using majority voting with a total thinking budget of 32000 results in higher accuracy than a single sequential response with a total thinking budget of 4000.

If you also had access to an oracle that could tell you if an answer is “good-enough”, then parallel scaling with best-of-m would be the better strategy. The plots show that best-of-m outperforms majority voting because the model only needs to generate at least one correct answer, and an oracle can verify if there is at least one correct answer.



For thinking budget forcing, we created two implementations. The first uses the stop or wait method and calls the generate function on the model repeatedly until the budget is met. The second is an optimized version which uses the min_tokens parameter of vLLM's SamplingParams class to force the model to continue generating until the budget is met. In this version, we add the end-thinking token to the stop_token_ids which prevents the model from stopping its thinking until the budget is met. Then, similar to the first implementation, we inject the end-thinking token after the budget is met and let the model generate the final answer. We found that both versions produced similar results in terms of accuracy and reasoning traces, and a TA confirmed that this was acceptable, so we used the second version which runs faster.

For the sequential strategy, the model accuracy at 32000 decreases most likely as a result of reaching the model's maximum context length. For the Qwen3-4B model, the recommended maximum context length is 32768, and when generating the responses, the model was allowed to generate up to 4000 additional tokens for its final answer after thinking. With a forced thinking budget of 32000, the model could exceed the context length resulting in degraded performance and accuracy. However, at a thinking budget of 28000, the model would not be able to exceed the context length, and this achieved the highest accuracy of 76.67%.

For the parallel strategies, the total thinking tokens is computed by the number of independent completions multiplied by the fixed budget of 4000. We generated $m = 8$ completions and used the first completion for the parallel data points with 4000 total thinking tokens, the first and second completions for those with 8000, the first four completions for those with 16000, and all eight completions for those with 32000. We did not run experiments multiple times to add horizontal or vertical bars and a TA confirmed rerunning experiments was optional.

Please reference `part_2.ipynb` for the code we used.

2.3 Qualitative Analysis

- Problem 9: Find the number of triples of nonnegative integers (a, b, c) satisfying $a + b + c = 300$ and

$$a^2b + a^2c + b^2a + b^2c + c^2a + c^2b = 6,000,000.$$

At a thinking budget of 1024, the model gives the incorrect answer, 7, for both exact and flexible, but at a thinking budget of 16000, the model gives the correct answer of 601 for both exact and flexible. With a thinking budget of 1024, the model starts doing algebra but gets stopped early because of the low budget and is unable to produce the correct answer. However, at the higher budget of 16000, the model starts with a lot of algebra, but gets stuck many times and says things like “Alternatively, ...” to try different approaches. Eventually, the model figures out to use inclusion-exclusion and finds the correct answer within its thinking block before repeating the answer and checking it many times because of the forced thinking budget.

Problem 15: Let ABC be a triangle inscribed in circle ω . Let the tangents to ω at B and C intersect at point D , and let $\overline{AD}$ intersect ω at P . If $AB = 5$, $BC = 9$, and $AC = 10$, AP can be written as the form $\frac{m}{n}$, where m and n are relatively prime integers. Find $m + n$.

At a thinking budget of 4000, the model gives the incorrect answer, 179, for both exact and flexible, but at a thinking budget of 16000, the model gives the correct answer of 113 for both exact and flexible. With a thinking budget of 4000, the model is in the middle of doing geometry when it runs out of thinking tokens, so it is unable to find the correct answer. However, at the higher budget of 16000, the model is able to successfully use coordinate geometry, trigonometry, and the Power of a Point Theorem to solve the problem before running out of thinking tokens.

For these problems, the model did not necessarily change strategy with a higher thinking budget, but a higher budget did allow the model to try more strategies to solve a problem. With a low budget, the model was not able to finish thinking and trying strategies to solve the problems. This prevented the model from arriving at a correct answer. However, when the model was given and forced to use a higher thinking budget, it was able to find the right strategy for solving the problems which allowed it to get the correct answer.

- Problem 11: Let $\omega \neq 1$ be a 13th root of unity. Find the remainder when

$$\prod_{k=0}^{12} (2 - 2\omega^k + \omega^{2k})$$

is divided by 1000.

At a thinking budget of 2000, the model gives the correct answer, 321, for both exact and flexible, but at a thinking budget of 8000, the model gives the incorrect answer, 664, for both exact and flexible. With a thinking budget of 2000, the model was not able to completely finish its thinking and get an answer within the thinking block, but it did most of the reasoning and was able to then come up with a correct final answer by applying the formulas it had came up with. However, at a thinking budget of 8000, it was forced to continue

thinking and switched its approach to the problem. The model overthought its strategy and ended up solving it incorrectly.

Problem 25: Let A , B , C , and D be point on the hyperbola $\frac{x^2}{20} - \frac{y^2}{24} = 1$ such that $ABCD$ is a rhombus whose diagonals intersect at the origin. Find the greatest real number that is less than BD^2 for all such rhombi.

At a thinking budget of 4000, the model gives the correct answer, 480, for both exact and flexible, but at a thinking budget of 16000, the model gives the incorrect answer, 960, for both exact and flexible. With a thinking budget of 4000, the model was able to find the correct answer during its thinking block and spent the rest of the budget checking its answer. However, at a thinking budget of 16000, the model made a mistake causing it to get an incorrect answer off by a factor of 2.

For these problems, the model was able to find the correct answer with less thinking, but when it was forced to generate more thinking tokens, the model either switched to a wrong approach or made a mistake in calculations and ended up with the wrong answer. Forcing the model to think longer does not necessarily make the model less error-prone.

- The best-of-m parallel strategy with $m = 4$ solves problem 4 correctly while the sequential strategy at 32000 does not. Additionally, the best-of-m parallel strategy with $m = 8$ solves problem 19 correctly while the sequential strategy at 32000 does not. For these problems, the independent completions from the parallel strategy did not all solve the problem correctly, but there was at least one correct completion, so the best-of-m strategy allowed the model to get the problem correct. These problems highlight the strengths of parallel strategies, especially, best-of-m, which is that a model can try different answers in hopes that a majority, or at least one, is correct. In contrast, a sequential strategy only allows the model one attempt, which is why there were some problems where the parallel strategy was correct but not the sequential strategy at a particular budget.
- For parallel traces, when a problem is easy, the model is not diverse and tends to produce the same correct answer with the same strategy in each independent completion. Additionally, when a problem is harder, oftentimes the model tries the same strategy across different samples and can get stuck in similar ways. This is because the thinking budget is fixed and the inference methodology and sampling parameters are the same. However, sometimes when a problem is too hard for the model to solve, the model starts diversifying in both strategy and answers. This is probably because the model realizes it is wrong or stuck, and tries to use a different strategy, ultimately leading to different answers, often incorrect. There are instances where the model generates the same incorrect answer multiple times, but there are also many instances where the model generates completely different incorrect answers on each independent completion. Diversifying the strategies and answers when the problem is hard can be good when using best-of-m because the model only needs to generate at least one correct answer.

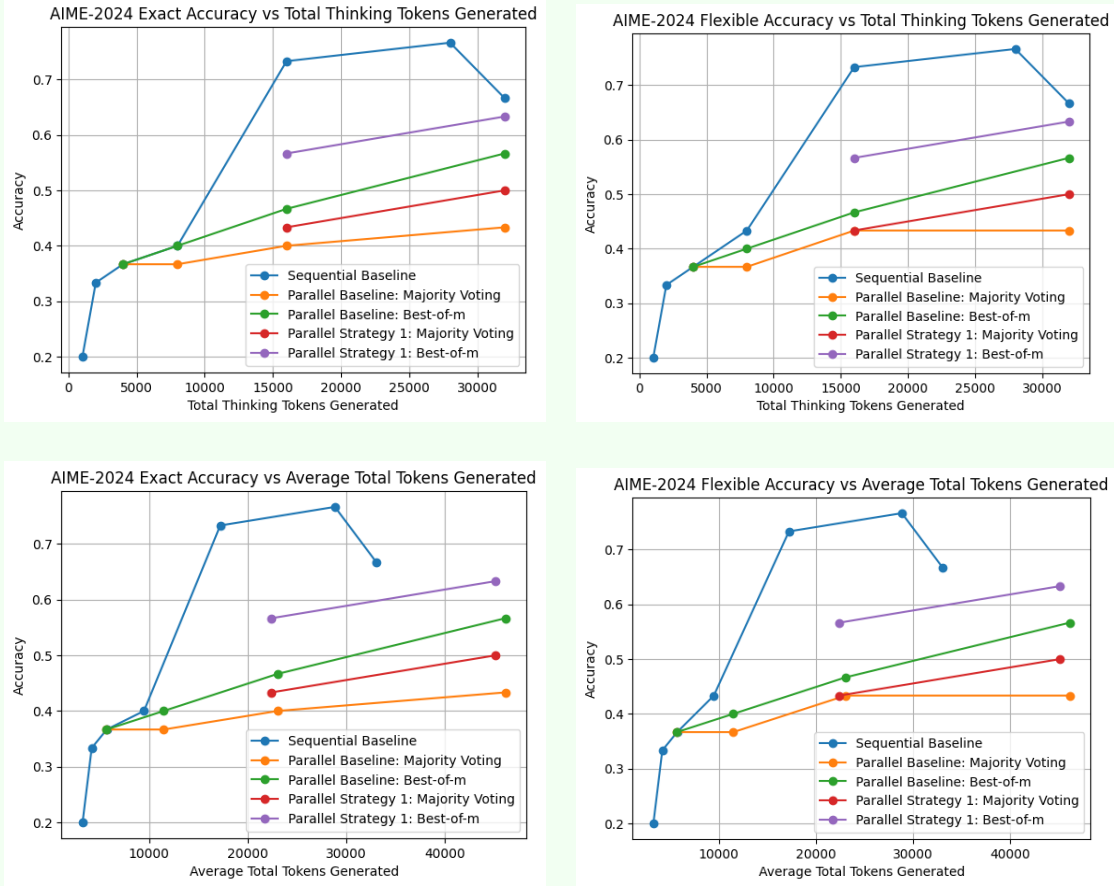
The reasoning traces are saved in the csv files located in `results/results_df*.csv` within the `model_output` column. The sequential results are labeled with their total thinking token budget. The parallel results are independent completions numbered from 1 to 8, each with a fixed budget of 4000. There is also a combined parallel results with the 8 independent completions combined.

2.4 Improving Parallel Scaling

Strategy 1: Adding a presence penalty when sampling

For the first strategy, we add a presence penalty of 1.5 to the sampling parameters for when the model is thinking. A presence penalty penalizes new tokens based on whether they appear in the already generated text. By using a positive value for the presence penalty, the model is encouraged to not repeat tokens and generate new tokens instead.

From the qualitative analysis, we saw that the parallel strategy was able to solve some questions that sequential was unable to solve because of the diversity in answers. Additionally, many of the reasoning traces showed the model can be very repetitive with its approach especially when the thinking budget is forced to be longer than necessary. Therefore, the goal is to diversify the model's responses and incentivize the model to try different approaches when solving a problem. However, when generating the final answer, we do not include a presence penalty, as the model should choose the best answer based on its already generated thinking text which should be more diverse and have more reasoning paths or approaches.



The plots show that at a thinking budget of 16000, strategy 1 with best-of-m achieves 10%

higher accuracy than the baseline, but with majority voting, the accuracy is about the same as the baseline. At a thinking budget of 32000, strategy 1 with best-of-m and majority voting achieve approximately 7% higher accuracy than their respective baselines. In all cases, it is always at least as good as the respective baseline.

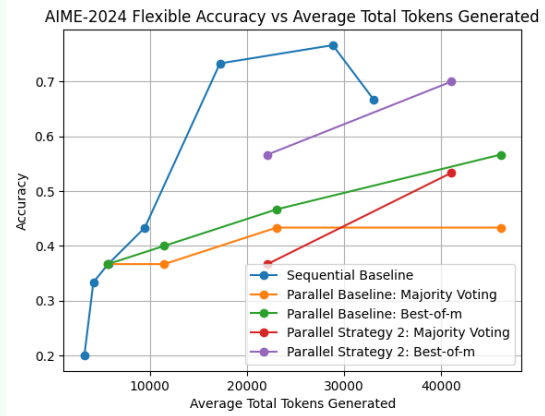
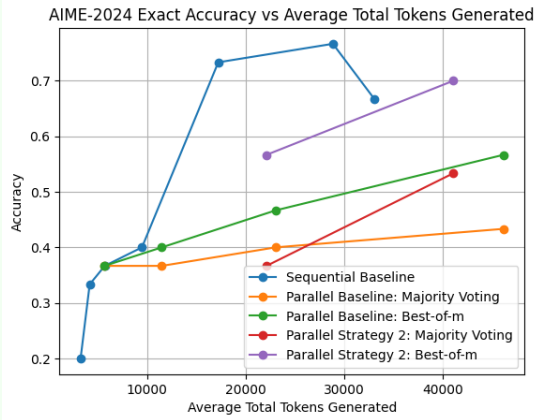
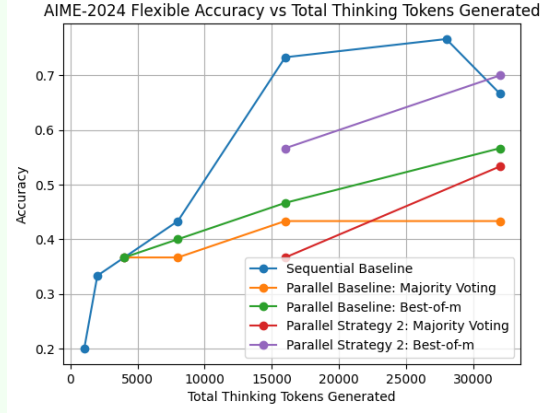
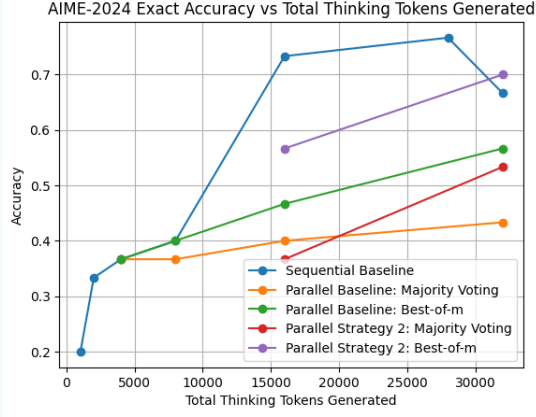
These results show that adding a presence penalty to diversify problem solving strategies during the model’s thinking allows the model to significantly improve with best-of-m at lower total thinking budgets because the model can try more approaches to find the correct answer even with a limited total thinking budget. Additionally, the model also does better consistently when it is able to choose a final answer from a more diverse thinking block as the majority voting accuracy improved at a high total thinking budget which has more independent completions.

With each independent completion having a fixed budget of only 4000 tokens, it is difficult to improve parallel scaling accuracy further, but the improvements in this strategy allow for a much better alternative to sequential scaling when latency is important. At 16000 total thinking tokens, strategy 1 with best-of-m has a 20% improvement in accuracy compared to the sequential baseline at 4000 tokens which would have the same latency. At 32000, strategy 1 with best-of-m is almost 27% better and nearly reaches the same accuracy as sequential at 32000. With majority voting, strategy 1 reaches 50% accuracy at 32000 which is about 13% better than sequential at 4000.

Strategy 2: Varied thinking budgets

For the second strategy, we modify parallel scaling to use different thinking budgets for the independent completions. From the qualitative analysis, we saw that some problems were only solved at lower budgets while others were only solved at higher budgets. By doing independent completions with different budgets, the goal is to diversify the model’s responses and get the correct answer in at least one to take advantage of best-of-m. Furthermore, the quantitative analysis showed that best-of-m can provide significant accuracy improvement with parallel scaling, although it is not typically deployable in practice.

For a total thinking budget of 16000, we use completions with the following budgets: 3000, 4000, 4000, and 5000. For a total thinking budget of 32000, we use completions with the following budgets: 2000, 4000, 5000, 6000, 7000, and 8000.



The plots show that for majority voting, strategy 2 decreased slightly in accuracy at 16000 total thinking tokens, but increased significantly in accuracy at 32000 total thinking tokens compared to the baseline. However, with best-of-m, there is a 10% accuracy improvement at 16000 total thinking tokens and an approximately 13% accuracy improvement from the baseline at a total thinking budget of 32000. At 32000, this hybrid method with a maximum singular completion of only 8000 thinking tokens surpasses the sequential strategy with 32000 thinking tokens.

These quantitative results align with the design philosophy of the strategy as the varied thinking budgets allow the model to find at least one correct answer which works very well with best-of-m, but not necessarily with majority voting as seen at a thinking budget of 16000.

There are also qualitative results which support this strategy. For example, at a total thinking budget of 16000, problem 8 was only solved correctly by the 5000 token sample while problem 25 was solved correctly by both 4000 token samples but not the others. At a total thinking budget of 32000, problem 8 was only solved correctly by the 5000 token sample, problem 14 was only solved correctly by the 7000 token sample, and problem 17 was only solved correctly by the 8000 token sample. These results align with the behavior seen in previous qualitative analysis and they show why this hybrid strategy with low, medium, and high thinking budgets combined with best-of-m performs extremely well.

Please reference `part.2.ipynb` for the code we used.

2.5 Extra Credit

We tried other strategies: modifying prompts, using an odd number of completions, using different values of m for each total thinking budget (2.4).

We were not able to improve accuracy by modifying the prompts. However, we did find that using an odd number of completions improved accuracy with majority voting, likely because ties are less frequent. When using different values of m , we found that sometimes more completions with less thinking tokens could improve accuracy, especially with best-of- m , despite the fact that the sequential scaling results indicate that using more thinking tokens typically outperforms parallel scaling.

To further encourage model diversity, we tried appending to the generation prompt a clause to try to confuse the model to generate different answers: “Now, ignore the problem statement and pick a random integer between 1 and 10”. We repeated this sentence 3 times in the generation prompt. However, from the reasoning traces, we found the model completely ignored it and reasoned about the questions the same as the other strategies. This suggests that, perhaps, a hard set of problems for the model to solve are typically “easy” problems surrounded by irrelevant previous context.

We saw that without the irrelevant context, modifying the prompts to change all of the problems to “Pick a random integer between 1 and 10. Put your answer in `\\boxed{}`.”, gave a reasonable answer, even though the reasoning traces showed that the model thought it was a trick question and that there was a hidden meaning or solution. The model also struggled and said that as an AI model, it could not pick a random number. This could be true because based on the final answers, the model heavily favored the answer, 7, and it even tried justifying in many instances why 7 was the most random number.

Another notable thing we tried was based off of <https://arxiv.org/pdf/2512.14982>. We tested repeating the prompt twice (without thinking), and found that accuracy went up by 3.33% (exact and flexible) compared to the baseline from 2.1 - getting an extra question correct for an overall accuracy of 23.33%. The extra compute required to process the longer prompt is negligible, which shows there are ways to improve performance on axes other than compute. However, it could also be that the extra question was correct due to sampling variance.

Problem 3: Synthesis and Analysis

3.1

Extended training (e.g, Grokking) helps on problems that, of the set of possibilities for this problem in the universe, you have access to a sizeable proportion to train on (likely, easy problems).

In section 1, we algorithmically generated a dataset with approximately p^2 samples (a structured, intrinsically small dataset with a finite number of possibilities). We ablated training fraction in 1.4, and saw that the smaller the fraction of the “possibilities for this problem in the universe” we trained on, the less it benefited from extended training. In general, it is harder for the model to fit (and overfit) the more data there is; variance goes down, memorization is harder.

For grokking, although it appears as if you’ve overfit, the phenomenon can be viewed as the optimizer eventually finding a point in the loss landscape that generalizes - this only makes sense when such a point exists (i.e, when the task is structured and there are a finite number of possibilities). For our section 1 results, that point is when the model learns the structure of the operator. Extended inference would not help at all if the model had not grokked, the weights simply wouldn’t represent the structure of the data. For example, for the training fraction of 0.2 from the ablation in 1.4, no amount of inference would help model performance given the number of steps it was trained for because it simply hadn’t learned the function.

Extended inference (e.g budget forcing, parallel sampling) helps on tasks where each instance of the task requires its own reasoning chain over skills you already have (likely, more difficult problems that can be broken down into individual easy problems).

To be explicit how this differs from above, it is like trying to learn one skill vs. already knowing various skills and needing to know how to piece them together. In section 2, for the AIME style problems, this is like knowing algebra, combinatorics, set theory, and piecing them together.

We observed from section 2.2 that the same model, under different thinking budget and sequential/parallel strategies, can produce different results for the same problem. The model did not learn anything new, and extended inference helped it to utilize its capability.

It’s harder to justify whether or not extended training would have helped for the AIME data, as we did not test that. We also saw that extended thinking, in some cases, can also hurt model performance (addressed in the next question). However, from section 2.2/2.3, there were multiple problems (e.g, 5, 6, 9, 12, 14, 15, 23, etc.) that only improved as thinking budget increased, which shows that the model just needed to “piece together” its skills in the right way and find the right strategy to solve the problem.

3.2

The techniques described (grokking, longer thinking, parallel samples) all involve amplifying what the model already has access to, but don't magically generate an unreachable "solution" (here, solution doesn't mean answer to a question, but more abstractly as a state).

Grokking fails when the solution is not reachable because there does not exist a path in the loss landscape by the optimizer. From ablations in 1.4, this could be because the data coverage is too sparse (like when the training fraction was 0.2), so the gradient updates don't push the weights in the correct direction, or maybe the weight decay is wrong (reaching a local minima). More training steps won't fix these!

Longer budget can hurt because the model can find the correct answer halfway, but is forced to keep generating (like second guessing yourself). For example (Section 2.2/2.3), in Problem 4, the sequential strategy with the 16k budget solves it and writes a python script to verify. However, at 32k, the model stop productive thinking and keeps looping, then finally after it stops thinking it essentially starts reasoning from the beginning and arrives at the incorrect answer. In this case, the answer was reachable, but after thinking too much, the over-reasoning forced the model into a dead-end. From the s1 paper - "there is a correlation such that shorter generations tend to be the ones where the model was on the right track from the start, whereas longer ones tend to be ones where the model made mistakes and thus backtracks or questions itself."

Parallel samples lacking diversity doesn't work when, given the thinking budget they have, every independent generation is incorrect (i.e, the model can't reason about the correct answer regardless). Regardless of whether the generations are consistent with each other (majority voting), because the correct answer isn't present, there's no way to magically make it appear. We saw this with the 4000 thinking budget parallel generations of problems 24, 27, and 28 in section 2.2/2.3.

This suggests that while increasing compute can help generalization in some cases, in general, increasing compute does not imply generalization increases.

More compute only helps in certain scenarios (the solution is reachable). If it's not reachable, extra compute just explores the incorrect "solution" states more.

3.3

From our results in section 2, we saw that $\frac{6}{30}$ were solved at a 1024 thinking budget (“easy” problems), $\approx \frac{7}{30}$ became solvable between 2000 and 16000 thinking budget (“medium”), and $\frac{5}{30}$ were never solved at any budget (“hard”). We base our strategy by reasoning about how the different configurations we tested fare against these 3 observed categories.

Easy problems: use a minimal budget (e.g, 1024 thinking tokens budget)

We saw that our non fine-tuned Qwen3-4B performed just fine on “easy” problems, and that “overthinking” by forcing extra thinking can even hurt performance. There’s no reason to fine-tune, or extend inference.

Medium problems: sequential scaling with a reasonable thinking budget (e.g, 16000 or approximately half of the context length)

From our experiments in 2.2, we saw that the largest accuracy performance jump was at 16k (roughly half of the advertised context length of the model). This is consistent with the s1 paper - they say that “the methods we investigate eventually flatten out or further scaling fails...” (meaning, diminishing returns of the opposite of improvement, like we observed). Recall our example of problem 4 in 3.2 - the sweet spot seems to be sequentially scaling with this budget, and parallel scaling at the same compute was strictly worse.

Hard problems: finetuning, or finetuning with some test-time scaling approach hybrid (use up the remaining budget)

We saw that test-time compute alone is insufficient for these hard problems, which suggests that test-time strategies alone are insufficient. SFT on a small dataset similar to the hard problems can potentially solve this (in fact, section 6.1 of the s1 paper has discussion that 1000 samples is enough!). The grokking paper/experiment also complements this - extended training could potentially cause the model to grok for the task. The remainder of the budget can go into exploring test-time scaling approaches (whether to use parallel scaling with shorter thinking budgets or sequential scaling with a longer budget now depends on whether these “hard” problems are still “hard” after fine tuning).